

# Runbook — Confluent Platform + Flink on Minikube

---

End-to-end operational guide for running `confluent-kafka-isotope` locally on **Minikube**: cluster → Kafka/Confluent Platform → Flink → SQL reports → traffic → observe → teardown. Every step maps to a target in the [Makefile](#), which is the source of truth.

This is the **Confluent Platform (self-managed) on Minikube** path. The Confluent Cloud for Apache Flink (CCAF) path is entirely different — Terraform-driven via `make cc-flink-reports-up`, no Minikube — see [README §4.5](#).

## Contents

- [0. Prerequisites](#)
  - [1. Cluster + Confluent Platform](#)
  - [2. Flink](#)
  - [3. Port-forward Kafka](#)
  - [4. Deploy the Flink SQL reports](#)
  - [5. Drive traffic](#)
  - [6. Observe](#)
  - [7. Teardown](#)
  - [Minimal path](#)
  - [Troubleshooting](#)
- 

## 0. Prerequisites (once per machine)

```
make install-prereqs    # docker, kubectl, minikube, helm, gettext,
gradle, openjdk17
make check-prereqs     # verify they're on PATH
```

Default Minikube sizing (override via env): `MINIKUBE_CPUS=6`, `MINIKUBE_MEM=20480`, `MINIKUBE_DISK=50g`. The node architecture is auto-detected so the right `cp-flink` image (amd64/arm64) is selected.

## 1. Cluster + Confluent Platform

```
make cp-up              # = check-prereqs → minikube-start → operator-
install → cp-deploy
make cp-watch           # watch pods come up (Ctrl+C to exit); or: make
cp-status
```

`cp-up` boots Minikube, installs the CFK (Confluent for Kubernetes) operator, and deploys Kafka (KRaft) + Schema Registry + Connect + ksqlDB + REST Proxy + Control Center into the `confluent` namespace.

`cp-up` deliberately does **not** bring up Flink — run [step 2](#) separately.

## 2. Flink

```
make flink-up          # cert-manager → operator → MinIO → CMF 2.4 → env
→ RBAC → app image
make cmf-status        # (~5 min the first time)
```

This installs cert-manager, the Confluent Flink Kubernetes Operator, **MinIO** (S3-compatible store for CMF artifacts), **CMF 2.4** (with artifact management + the writable environment catalog enabled), creates the `dev-local` Flink environment, applies the Flink RBAC, and builds the custom `cp-flink` 2.1 image (`isotope-cp-flink-sql:local`) that bakes in the Kafka + Avro SQL connectors and the S3 filesystem plugin.

**The reports run as a CMF *Application*, not SQL statements.** CMF's SQL-statement runtime (`io.confluent.flink.FlinkCompiledPlanExecutor`) ships only in the `cp-flink-sql` image, which exists only at **Flink 1.19** — and two reports are `ProcessTableFunctions`, a **Flink 2.x** feature. So all seven reports deploy as a single Flink 2.1 CMF **Application** (entry point `IsotopeReportsJob`), which runs the same `.fql` and surfaces in CMF / Control Center as a managed application. No raw session cluster is created.

**CMF license.** The `cp-cmf` image ships with a date-locked trial license. Once it expires, the CMF pod `CrashLoopBackOffs` on startup (`LicenseInitiator` throws) and `make flink-up` fails at the CMF readiness wait. To run past expiry, supply a Confluent license via a secret and point `CMF_LICENSE_SECRET` at it:

```
kubectl create secret generic confluent-license-for-cmf -n confluent \
  --from-file=license.txt=/path/to/license.txt # key MUST be
license.txt
make flink-up CMF_LICENSE_SECRET=confluent-license-for-cmf
```

Export `CMF_LICENSE_SECRET` in your shell to make plain `make flink-up` pick it up.

## 3. Port-forward Kafka

```
make kafka-pf-up      # localhost:30092 → Kafka, localhost:8081 → SR
```

Prereq for everything the host-run gradle app does — `App.java`'s defaults already point at `localhost:30092` / `localhost:8081`. Stop with `make kafka-pf-down`.

## 4. Deploy the 7 Flink reports (as a CMF Application)

```
make flink-reports-up      # builds the app shadow JAR (:ptf:shadowJar) →
pre-creates                # 4 source + 7 sink topics → uploads the JAR as a
cmf://                     # artifact (MinIO) → deploys the FlinkApplication
→ waits RUNNING
```

All seven reports run in **one** Flink 2.1 CMF Application (**isotope-reports**): five pure Flink SQL plus two JAR-backed **ProcessTableFunctions** (**LatencyPercentilesPTF**, **StuckTracePTF**), executed together as a single **StatementSet** by **IsotopeReportsJob**. Sink topics use Apache Flink's **avro-confluent** format — SR-framed Avro, auto-registered on first write — so Control Center renders the rows natively. The application appears in CMF's applications API and Control Center's Flink tab. Drop everything (application + artifact + sink topics) with **make flink-reports-down**.

## 5. Drive traffic (required to see report rows)

All report jobs aggregate over **TUMBLE(event\_time, INTERVAL '1' MINUTE)** windows, which only emit when the watermark advances past **window\_end**. Spread records across **multiple** windows:

```
# 30 records, 5s apart ≈ 2.5 min of event-time → spans 3+ windows
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep
5; done
```

Or run the pipeline-position stages, each in its own terminal:

```
./gradlew :app:run --args="place hello"    # origin → orders.placed
./gradlew :app:run --args="enrich"         # orders.placed →
orders.enriched
./gradlew :app:run --args="fulfill"        # orders.enriched →
orders.fulfilled
./gradlew :app:run --args="ship"           # terminal consume
orders.fulfilled
```

Wait ~90s after the **last** record before checking results — that's the watermark catching up. **stuck\_trace\_alerts\_1m** only fires for a trace that goes ≥60s of event time without a fresh hop; to exercise it, send one record to **orders.placed**, skip the **enrich/fulfill** hops, and keep sending unrelated records so the watermark crosses **event\_time + 60s**.

## 6. Observe

```
make c3-open              # Control Center — report topics render natively
(Avro+SR)
make flink-sql            # interactive Flink SQL client inside the
JobManager pod
```

```
make flink-ui          # Flink job graph / watermarks
make metrics-up       # optional: Prometheus + Grafana for the stateless
meters
```

The metrics showcase has its own runbook — [k8s/monitoring/README.md](#) (and the meter/PromQL reference in [docs/metrics.md](#)). Stop the background port-forwards with `make c3-stop` / `make flink-ui-stop` / `make metrics-down`.

## 7. Teardown

Pick the depth:

```
make flink-reports-down # drop reports / views / functions only
make metrics-delete     # remove the Prometheus/Grafana showcase (pods +
namespace)
make flink-down         # Flink cluster + CMF + operator + cert-manager
make cp-down           # CP + operator (keeps Minikube running)
make confluent-teardown # everything + stop Minikube
make nuke              # confluent-teardown + minikube-delete +
uninstall-prereqs (factory reset)
```

## Minimal path (no Flink)

To just watch a trace propagate without the Flink SQL reports:

```
make cp-up
make kafka-pf-up
./gradlew :app:run --args="place hello" # then enrich / fulfill / ship
```

Flink ([step 2](#)) and the reports ([step 4](#)) are only needed for the SQL report topics.

## Troubleshooting

- **Pods stuck Pending.** Minikube is under-resourced — raise `MINIKUBE_CPUS` / `MINIKUBE_MEM` and `make minikube-delete && make cp-up`.
- **Flink job submission fails (services forbidden).** The supplemental RBAC didn't apply — `make flink-rbac`.
- **No report rows.** Almost always the watermark — see [step 5](#); traffic must span multiple 1-minute windows and you must wait ~90s after the last record.
- **App can't reach Kafka.** `make kafka-pf-up` isn't running, or the forward died — re-run it and confirm `localhost:30092` / `localhost:8081` are live.
- **Control Center's Flink tab is blank.** CMF proxy connectivity — `make cmf-proxy-inject` (and `make cmf-proxy-logs` to debug).
- **make flink-up times out waiting for the CMF pod / CMF CrashLoopBackOff.** Check `kubectl logs -n confluent -l app.kubernetes.io/name=confluent-manager-for-apache-`

`flink`. If you see `Trial license ... expired / LicenseInitiator ... Constructor threw exception`, the image's embedded trial license has expired — supply your own via `CMF_LICENSE_SECRET` (see the CMF license note in [step 2](#)). Wiping the CMF PVC does **not** reset it; the expiry is baked into the image.